Comparative Classification of Microcontroller Boards and Electronic Components with Convolutional and Transformer Networks

ITRI626 Practical Project: Deep Learning for Image Classification

Carlo Barnardo (43870449) Itumeleng Ntela (43528309)
Karl Emmanuel Edoun (51622114)

School of Computer Science and Information Systems, North-West University

November 3, 2026

Abstract

Recognising microcontroller development boards and discrete electronic components from photographs is useful for sorting electronic waste, keeping track of laboratory inventory and checking assemblies. We frame it as a 10-class single-label image classification problem: four popular development boards (Arduino Nano, ESP8266 NodeMCU, Heltec ESP32 LoRa and Raspberry Pi 3) and six component types (integrated circuits, capacitors, diodes, resistors, inductors and transformers). The images come from two public datasets: bench photographs of boards (MIT licence) and component images crawled from the Digi-Key catalogue. After cleaning, which removed corrupt files, exact duplicates and images whose label conflicted with a near-duplicate, and a visual review of every image, the dataset contains 1902 images. They are split 70/15/15 with a group-aware stratified procedure, so that photographs of the same physical object never appear in more than one split. We compare three architecture families on the same split: a small convolutional network trained from scratch, an ImageNet-pretrained ResNet-18 and an ImageNet-pretrained Vision Transformer (ViT-B/16). On the held-out test set the best model, ResNet-18 (transfer), reaches a macro-F1 of 0.901 and an accuracy of 90.2%. We analyse per-class behaviour, the most frequent confusions and the trade-off between accuracy, model size and inference time, and we discuss the limitations of the dataset.

1 Introduction and problem motivation

Microcontroller boards are the building blocks of embedded systems, Internet-of-Things devices and engineering education. A university laboratory or maker space can own hundreds of boards from different families, and recycling facilities receive mixed electronic waste whose value depends on what is in it. Identifying boards and components by hand is slow and error-prone, especially when boards from different families share a form factor or solder-mask colour. An image classifier that recognises the board or component in a photograph could support automatic inventory, sorting lines and visual checks during assembly.

The task is harder than it first appears. Several classes are *fine-grained*: the Arduino Nano and the ESP8266 NodeMCU are both small dark rectangular boards with two rows of header pins, and the ESP8266 and the Heltec ESP32 differ mainly in the module and the ESP32's OLED display. Among the components, inductors and transformers both show copper windings, and many diodes, resistors and capacitors are small cylinders with two leads. Images also vary in viewpoint, lighting, background and scale, from close-up product photographs to whole circuit boards. The four board classes are small (about 45 images each) while the component classes have several hundred, so the dataset is also imbalanced.

Deep convolutional neural networks (CNNs) have been the standard approach to image classification since AlexNet [11]. Residual networks [7] made very deep CNNs trainable, and Vision

Transformers (ViT) [5] showed that self-attention over image patches can match or beat CNNs when pretrained on large data. For small, specialised datasets like ours, *transfer learning*, i.e. starting from weights pretrained on ImageNet [4] and fine-tuning, is known to improve accuracy considerably over training from scratch [21, 9].

Aim and contributions. The aim of this project is to build and fairly compare three image classifiers from different model families for the 10 classes above, and to understand where and why they fail. Concretely we:

1. collect, clean and document an image dataset, with explicit near-duplicate detection and a group-aware split that prevents leakage (Section 2);
2. train a custom CNN baseline, a pretrained ResNet-18 and a pretrained ViT-B/16 under one shared, reproducible protocol (Sections 3–4);
3. report accuracy, macro- and per-class precision/recall/F1, confusion matrices, learning curves and inference cost on a held-out test set, and analyse the errors (Sections 5–6).

Scope. Each image contains one dominant object and receives exactly one label. Locating several components on a populated circuit board is an object-detection problem and is outside our scope (Section 6.3).

2 Dataset, preparation and ethics

2.1 Source and licence

The dataset combines two public sources. The script `src/mlproject/data/build_public_dataset.py` rebuilds `data/raw/` from them exactly.

- **Boards:** the microcontroller detection dataset of Tanner [1], released on GitHub under the MIT licence: 149 photographs, taken with a phone on a desk and annotated with bounding boxes for Arduino Nano, ESP8266, Heltec ESP32 LoRa and Raspberry Pi 3. We crop every box with 10% padding into one classification image (177 crops: 43, 47, 44 and 43). Crops from the same photograph keep a shared group id.
- **Components:** the electronic-component dataset of Faraji [2]: 4,038 images in six folders, collected by crawling the Digi-Key catalogue and from the Kaggle dataset "Electronic components and devices" [3]. To limit the imbalance against the board classes, we randomly sample at most 400 images per component class (seed 42); inductors have only 265.

The component repository has no licence file; we use its images for non-commercial coursework only and acknowledge it and its upstream sources in full.

2.2 Class definitions

Table 1 defines the classes. The four board classes are widely used hobbyist and teaching platforms and are the boards for which openly licensed photographs were available. The six component classes cover the most common discrete and integrated parts. The class set includes visually similar pairs (Nano vs. ESP8266, inductor vs. transformer), so the comparison tests fine-grained discrimination as well as coarse category recognition.

Table 1: Target classes.

Class	Definition / visual cues
`arduino_nano`	Compact ATmega328 board for breadboards, mini-USB, two rows of male header pins
`esp8266`	ESP8266 NodeMCU board; ESP-12 shielded module with PCB antenna, USB bridge chip
`esp32_heltec_lora`	Heltec WiFi LoRa 32 board: ESP32 with LoRa radio and a small OLED display
`raspberry_pi_3`	Raspberry Pi 3 single-board computer: green PCB, four USB ports, Ethernet, HDMI
`ic`	Integrated circuits: packaged chips (DIP, SOIC, QFP, BGA), sometimes shown mounted
`capacitor`	Electrolytic, ceramic, film and super-capacitors
`diode`	Rectifier, signal, Zener and light-emitting diodes, through-hole and SMD
`resistor`	Axial colour-band and SMD resistors
`inductor`	Wire-wound coils, toroids and SMD inductors
`transformer`	Mains and signal transformers with laminated or toroidal cores

2.3 Labelling and quality control

Board labels come from the bounding-box annotations of the source dataset; component labels come from the source folders, which follow the Digi-Key product categories. Exact and near-duplicate copies with conflicting labels were removed automatically (Section 2.4). Catalogue categories turned out to be noisy, so every image in all ten classes was inspected visually on numbered contact sheets. 509 images were excluded; each exclusion and its reason is listed in `data/manual_exclude.csv`, and the preparation script applies the list automatically. The decision rule was that the labelled object must be the clear subject of the image. The main reasons were: images of whole circuit boards, computers or scenes in which no single component dominates (most frequent in `ic`, which lost 218 of 400 images); mains transformers and historic induction coils filed under `inductor`, which would directly confuse it with `transformer`; multi-pin IC packages filed under `diode`; and power-grid scenes, UPS units and rack equipment filed under `transformer`. The four board classes needed no exclusions. No image was relabelled. The first visual pass over the contact sheets was done with an AI assistant (Claude, Anthropic), which proposed the exclusions and reasons.

2.4 Automatic cleaning

After the manual exclusions, the preparation script (`src/mlproject/split.py`) cleans the remaining images in three automatic passes and writes every removal and its reason to `data/splits/cleaning_report.js`

1. **Corrupt or unusable files.** Images that cannot be decoded by PIL, are smaller than 1 kB or have a side shorter than 32 pixels are removed (0 images).
2. **Exact duplicates.** Byte-identical files are found by MD5 hash. One copy is kept; if copies carry different labels the label is ambiguous and all copies are removed (10 images).
3. **Near duplicates.** Each image gets a 256-bit difference hash (dHash [10]): the image is converted to grey, resized to 17×16, and adjacent pixels are compared. Two images whose hashes differ in at most 10 bits are treated as near duplicates (the same photo resized, recompressed or slightly cropped). Near duplicates in *different* classes indicate an uncertain label and are removed (19 images). Near duplicates in the *same* class (1954 pairs) are kept but merged into one group, so they always land in the same split (Section 2.6).

In total 2440 raw images were reduced to 1902. Table 2 gives the counts per class before and after cleaning and per split.

Table 2: Images per class before and after cleaning, and per split (seed 42).

Class	Raw	Cleaned	Train	Val	Test
arduino_nano	43	43	31	7	5
capacitor	400	355	259	53	43
diode	400	291	214	42	35
esp32_heltec_lora	44	44	32	5	7
esp8266	47	47	36	8	3
ic	400	182	133	26	23
inductor	265	181	132	27	22
raspberry_pi_3	43	43	31	5	7
resistor	398	382	300	42	40
transformer	400	334	246	49	39
Total	2440	1902	1414	264	224

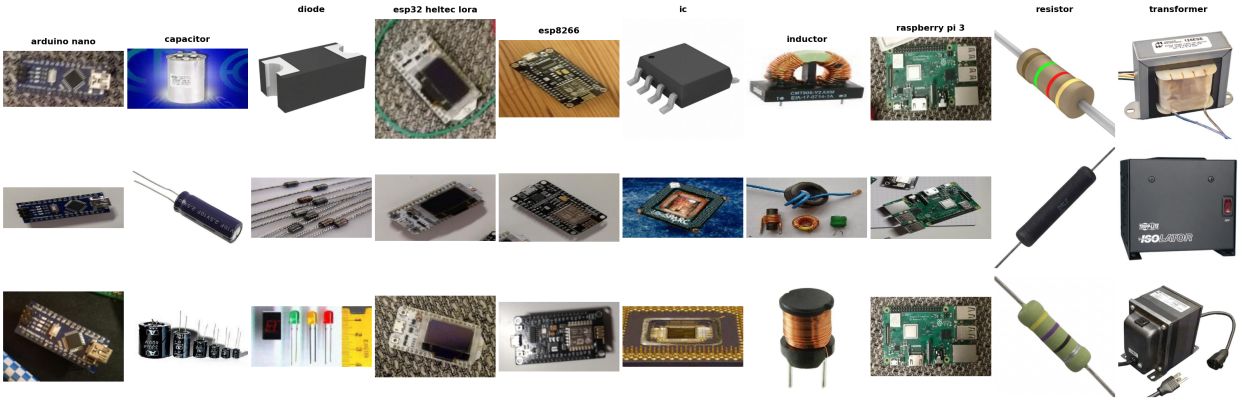


Figure 1: Representative training images from every class (random sample, seed 42).

2.5 Class imbalance

The data set is clearly imbalanced: the largest class (`resistor`, 382 images) has about nine times as many images as the smallest classes (`arduino_nano` and `raspberry_pi_3`, 43 each), and all four board classes have fewer than 50 images, against 181–382 for the component classes. We handle imbalance in two ways. First, the cross-entropy loss is weighted by inverse class frequency, $w_c = N/(C\,n_c)$, where N is the number of training images, C the number of classes and n_c the count of class c, so every class contributes equally to the expected loss. Second, we use macro-averaged F1 and balanced accuracy as headline metrics, because plain accuracy is dominated by the large classes. We did not oversample, because repeated images would increase the risk of overfitting on the smallest classes.

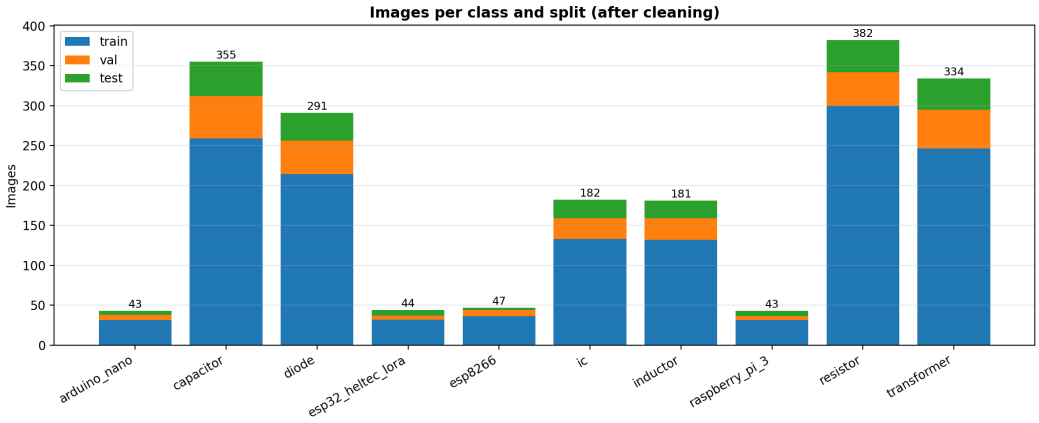


Figure 2: Images per class and split after cleaning.

2.6 Training, validation and test split

A random split of individual images would leak information if several photographs show the same physical board, because the model could memorise that specific board rather than learn the class. We therefore assign every image a *group id*: images of the same physical object share a group, encoded in the file name (`board07__3.jpg`) or a sub-folder, and same-class near duplicates are merged into a common group automatically. The data is then split with scikit-learn's `StratifiedGroupKFold` [15] into $k = 7$ folds with random seed 42. One fold is the test set, one the validation set and the remaining five form the training set, which gives approximately 70/15/15 while keeping class proportions similar in each split. The script asserts that no group id and no file hash occurs in more than one split. The resulting split contains 1414 training, 264 validation and 224 test images in 1764 groups. The split files (`data/splits/{train,val,test}.csv`) are submitted, and `python -m mlproject prepare` recreates them exactly.

2.7 Ethical and legal considerations

The images show inanimate electronic hardware only; no people, personal data or health information are involved. Some photographs may contain manufacturer logos or silkscreen text, which are visible features of the products and are used only for classification research. The board dataset is MIT-licensed. The component images carry no licence file and were originally crawled from a public product catalogue; we use them for non-commercial coursework only and acknowledge both the repository and its upstream sources in the references.

3 Methodology

3.1 Overall workflow

The pipeline is implemented in PyTorch [14] and torchvision [19] as a Python package with a command-line interface. `prepare` cleans and splits the data; `train` trains one model and saves the checkpoint with the lowest validation loss; `evaluate` evaluates that checkpoint once on the test set; `report-assets` regenerates every table and figure in Sections 2–5 from the saved outputs. The test set is used only for the final evaluation, never for model selection.

3.2 Model 1: custom CNN baseline (trained from scratch)

The baseline is a plain four-stage CNN. Each stage is a 3×3 convolution followed by batch normalisation [8], ReLU and 2×2 max pooling, with 32, 64, 128 and 256 filters. Global average pooling and dropout ($p = 0.3$) [17] feed a linear classifier. With about 0.4 M parameters it measures

what can be learned from our data alone, without prior knowledge, and gives a lower bound for the benefit of transfer learning.

3.3 Model 2: ResNet-18 (residual CNN, transfer learning)

ResNet [7] stacks residual blocks that learn a correction $\mathcal{F}$ to their input,

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \{W_i\}) + \mathbf{x}, \tag{1}$$

so gradients can flow through the identity shortcut and deep networks remain trainable. ResNet-18 has 8 such blocks (11.2 M parameters). We load the torchvision ImageNet-1k weights and replace the 1000-way output layer with dropout ($p = 0.3$) and a new 10-way linear layer.

3.4 Model 3: Vision Transformer ViT-B/16 (transformer, transfer learning)

ViT [5] has no convolutions in its encoder. The 224×224 image is cut into $14 \times 14 = 196$ patches of 16×16 pixels; each patch is linearly projected to a 768-dimensional token, a learnable class token and position embeddings are added, and 12 transformer encoder layers [20] apply multi-head self-attention,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{2}$$

so every patch can attend to every other patch from the first layer on. The final class token is classified by a new dropout + linear head. ViT-B/16 has 85.8 M parameters and weak built-in spatial priors: it relies on large-scale pretraining for the locality and translation invariance that CNNs have by design [16]. This makes it a meaningfully different family from the two CNNs, and lets us test whether global attention helps with fine-grained classes, where the decisive details (a module shield, a jumper) are small.

3.5 Transfer learning strategy

For both pretrained models we fine-tune *all* layers rather than freezing the backbone, because our images (close-ups of hardware) differ from typical ImageNet photographs, and adapting low-level features usually helps under such a domain shift [21, 9]. To avoid destroying the pretrained features in the first updates, which happens when the new, randomly initialised head produces large gradients, we use a small learning rate with a linear warm-up [6]. All inputs are normalised with the ImageNet mean and standard deviation that the pretrained weights expect.

4 Experimental setup and reproducibility

4.1 Preprocessing and augmentation

All images are converted to RGB, resized to 224×224 and normalised with the ImageNet channel statistics ($\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$). The following augmentations are applied *only to the training set*; validation and test images are only resized and normalised:

- random horizontal and vertical flips ($p = 0.5$) and rotation up to $\pm 20°$: boards and components lie in any orientation on a bench, so orientation carries no class information;
- colour jitter (brightness, contrast and saturation $\pm 20\%$, hue ± 0.05): lighting varies between rooms, lamps and cameras; the hue range is kept small because colour (e.g. the green PCB of the Raspberry Pi) is a genuine cue;
- random translation up to 8% and scaling 0.92–1.08: objects are not perfectly centred or equally large in real photographs.

We did not use MixUp or CutMix, because blending two boards produces images that cannot occur in practice and could blur fine-grained cues. Random crops that might cut away the distinguishing part of a board were also avoided.

4.2 Training protocol and hyperparameters

All models use the same split, the same augmentations, AdamW [13] with weight decay 10^{-4}, class-weighted cross-entropy, a linear warm-up over two epochs followed by cosine decay [12] to 10^{-6}, and an *effective* batch size of 32. ViT-B/16 uses a physical batch of 16 with two gradient-accumulation steps to fit in GPU memory; this is mathematically equivalent to batch 32 for the gradient. Training stops early when the validation loss has not improved for 7 epochs, and the checkpoint with the lowest validation loss is evaluated. Mixed-precision (fp16) training is used on GPU.

Learning rates differ between models on purpose, and we justify each choice rather than tuning on the test set. The custom CNN starts from random weights and uses 10^{-3}, a standard rate for Adam-type optimisers. ResNet-18 is fine-tuned at 3×10^{-4} so that pretrained features change gradually. ViT-B/16 uses 5×10^{-5}, because transformers are known to diverge or forget pretraining when fine-tuned at CNN learning rates [18]. The CNN gets a larger epoch budget because training from scratch converges more slowly; early stopping decides the actual number of epochs. All settings were fixed before training and no hyperparameter was changed after looking at validation or test results. Table 3 lists the settings that were actually used, read from the training logs.

Table 3: Training configuration per model. All models: AdamW, input 224×224, linear warm-up then cosine decay, class-weighted cross-entropy, early stopping on validation loss, seed 42. Batch is given as batch×accumulation steps (effective batch 32 for all). Epochs: maximum (actually trained).

Model	Pretrained weights	Batch	Epochs	LR	WD	Params (M)
Custom CNN (scratch)	none (random init)	32×1	40 (38)	1e-03	1e-04	0.42
ResNet-18 (transfer)	ImageNet-1k (torchvision DEFAULT)	32×1	25 (25)	3e-04	1e-04	11.18
ViT-B/16 (transfer)	ImageNet-1k (torchvision DEFAULT)	16×2	25 (15)	5e-05	1e-04	85.81

4.3 Software and hardware

Python 3.13.16, PyTorch 2.11.0+cu130, torchvision 0.26.0+cu130, scikit-learn 1.6.1 on Linux 6.6.159+. GPU: Tesla T4, 14.6 GB; CPU: x86_64. The random seed 42 is set for Python, NumPy and PyTorch (including the data-loader shuffling), and cuDNN is run in deterministic mode. The complete environment is pinned in `uv.lock` and `devenv.nix`, and the README lists the commands that reproduce every result.

5 Results

5.1 Overall comparison

Table 4 compares the three models on the held-out test set of 224 images. The custom CNN reaches 50.9% accuracy (macro-F1 0.543), ResNet-18 90.2% (0.901) and ViT-B/16 87.9% (0.897). ResNet-18 achieves the best macro-F1 (0.901) and accuracy (90.2%), followed closely by ViT-B/16 (0.897, 87.9%); both are far ahead of the custom CNN (0.543, 50.9%). The ranking is the same by

accuracy, balanced accuracy and macro-F1. The gap between the two pretrained models is small: with 224 test images one image is worth about 0.45 percentage points, so ResNet-18's lead of about 2 percentage points corresponds to five images and the macro-F1 difference of 0.003 is within the noise of a single run. On the validation set the order was actually reversed (validation macro-F1 of the selected checkpoints 0.945 for ViT-B/16 versus 0.933 for ResNet-18), which confirms that the two models perform at essentially the same level on this data. The gap to the custom CNN (about 36 points of macro-F1) is, by contrast, very large and certainly not noise. In inference cost the models differ strongly: on the T4 GPU ViT-B/16 needs 12.1 ms per image, about eight times more than ResNet-18 (1.4 ms) and the CNN (1.1 ms).

Table 4: Held-out test results (identical test split for all models). Latency is mean forward-pass time per image on the hardware in Section 4; train time is wall-clock minutes.

Model	Acc (%)	Bal. acc (%)	Macro P	Macro R	Macro F1	W. F1	ms/img	Train (min)
Custom CNN (scratch)	50.9	64.6	0.515	0.646	0.543	0.500	1.07	6.8
ResNet-18 (transfer)	90.2	91.0	0.909	0.910	0.901	0.902	1.44	4.5
ViT-B/16 (transfer)	87.9	90.0	0.902	0.900	0.897	0.879	12.11	5.3

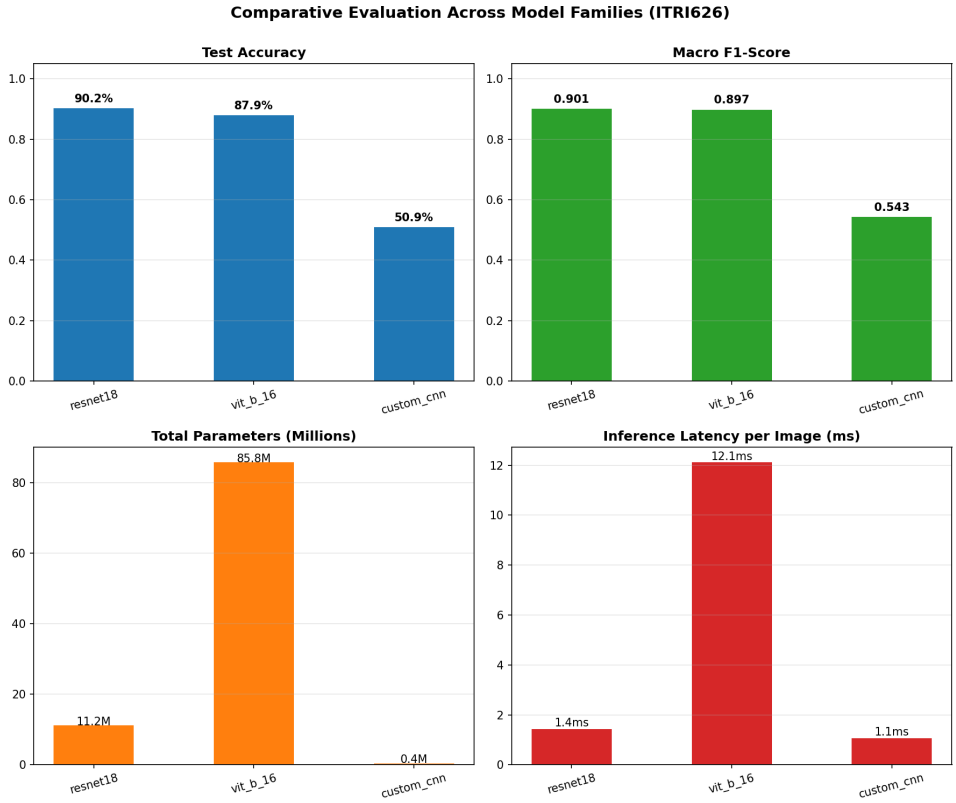


Figure 3: Test accuracy, macro-F1, parameter count and inference latency of the three models.

5.2 Learning curves

Figures 4–6 show training and validation loss together with validation accuracy and macro-F1 for each model, and Figure 7 overlays the validation curves. The pretrained models converge within a few epochs: after only two epochs ResNet-18 reaches a validation macro-F1 of about 0.80 and ViT-B/16 about 0.85, and they pass 0.90 at epoch 3 (ViT-B/16) and epoch 9 (ResNet-18). ViT-B/16 reached its lowest validation loss at epoch 8 and was stopped early at epoch 15; ResNet-18 kept improving slowly, peaked at epoch 20 and ran the full 25 epochs. Both pretrained models drive the training loss close to zero (final training accuracy 99–100%) while the validation loss levels

off around 0.2–0.3, so they fit the training set almost perfectly and the remaining generalisation gap is moderate rather than a sign of severe overfitting. ResNet-18 shows one validation spike at epoch 5, a typical transient while the learning rate is still near its maximum after warm-up. The custom CNN behaves completely differently: its validation curves are noisy, it needs 31 epochs to reach its best validation loss (1.27) and it stops at epoch 38 with a training accuracy of only 54%. Its training loss stays above 1.0, so the CNN is *underfitting*: with about 1,400 training images and random initialisation it cannot learn features that are discriminative enough, rather than memorising the training set.

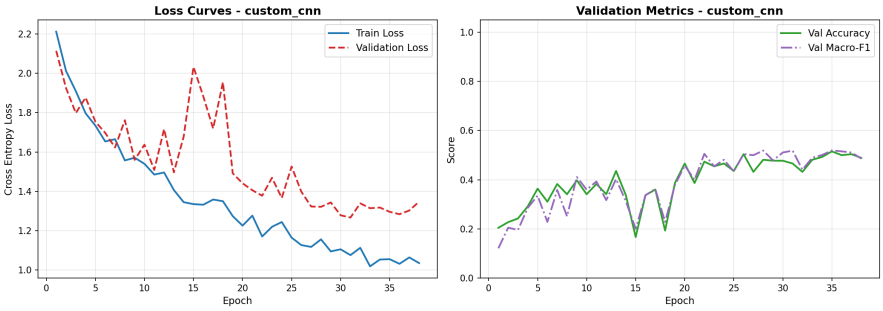


Figure 4: Learning curves: custom CNN.

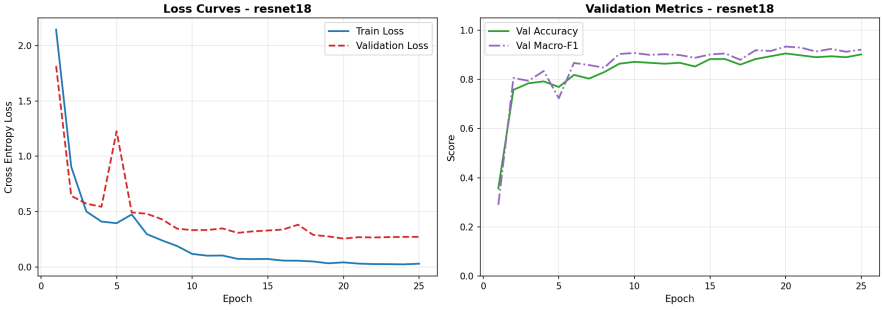


Figure 5: Learning curves: ResNet-18.

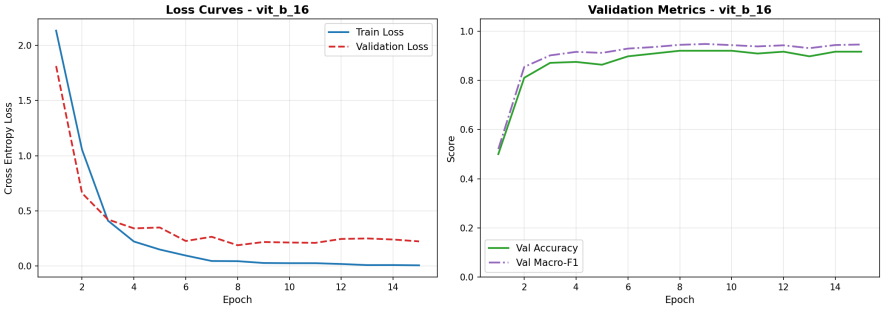


Figure 6: Learning curves: ViT-B/16.

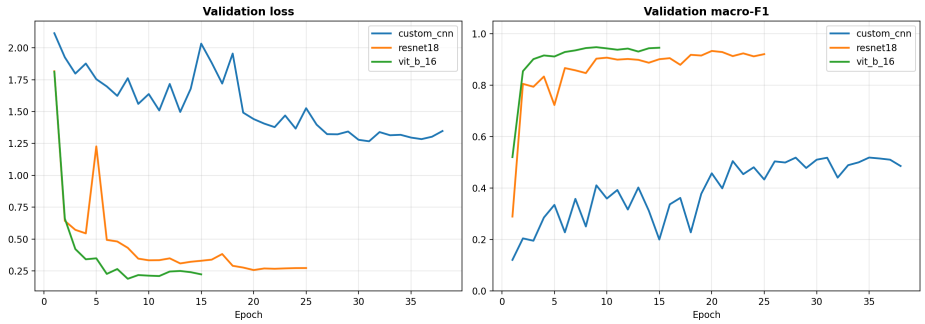


Figure 7: Validation loss and validation macro-F1 of all three models.

5.3 Per-class performance

Table 5 and Figure 8 give per-class precision, recall and F1. For both pretrained models the board classes are the easiest: `esp32_heltec_lora` and `raspberry_pi_3` reach F1 = 1.00 for both, and `arduino_nano` 0.89. This is partly because each board class shows a single physical board photographed in one setting (Section 6.3), so these scores are optimistic. Among the components, `transformer` (F1 0.96 for ResNet-18) and `resistor` (0.93) are the easiest: transformers are large and visually distinctive and resistors have the characteristic colour bands. The hardest classes are `diode`, `capacitor` and `ic`: ResNet-18 obtains F1 0.86, 0.86 and 0.85 and ViT-B/16 0.79, 0.81 and 0.91. For ResNet-18 the weakness of `ic` is in recall (0.74: some ICs are predicted as other classes), while its precision is 1.00. The lowest-precision class for ResNet-18 is `esp8266` (0.60) despite perfect recall: the few images that are wrongly predicted as `esp8266` weigh heavily because the class has only six test images. Class size alone does not explain difficulty, because the small board classes are the easiest; visual similarity between components matters more. The custom CNN is weak everywhere except `raspberry_pi_3` (F1 0.93) and `resistor` (0.70).

Table 5: Per-class test precision (P), recall (R) and F1. n = test images per class.

		Custom CNN (scratch)			ResNet-18 (transfer)			ViT-B/16 (transfer)		
Class	n	P	R	F1	P	R	F1	P	R	F1
arduino_nano	5	0.50	1.00	0.67	1.00	0.80	0.89	1.00	0.80	0.89
capacitor	43	0.53	0.37	0.44	0.84	0.88	0.86	0.83	0.79	0.81
diode	35	0.60	0.34	0.44	0.84	0.89	0.86	0.78	0.80	0.79
esp32_heltec_lora	7	0.45	0.71	0.56	1.00	1.00	1.00	1.00	1.00	1.00
esp8266	3	0.33	1.00	0.50	0.60	1.00	0.75	0.75	1.00	0.86
ic	23	0.37	0.43	0.40	1.00	0.74	0.85	0.95	0.87	0.91
inductor	22	0.28	0.41	0.33	0.95	0.86	0.90	0.90	0.82	0.86
raspberry_pi_3	7	0.88	1.00	0.93	1.00	1.00	1.00	1.00	1.00	1.00
resistor	40	0.63	0.80	0.70	0.90	0.95	0.93	0.91	0.97	0.94
transformer	39	0.58	0.38	0.46	0.95	0.97	0.96	0.90	0.95	0.93

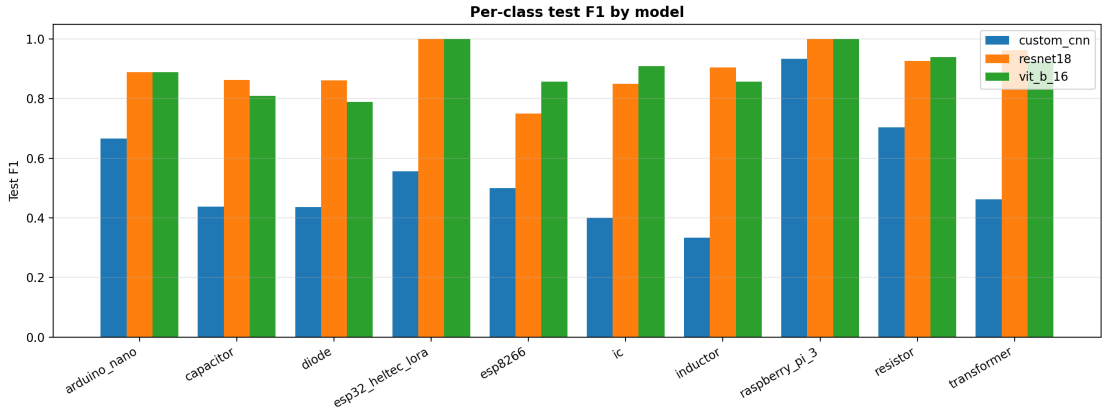


Figure 8: Per-class test F1 of each model.

5.4 Confusion matrices

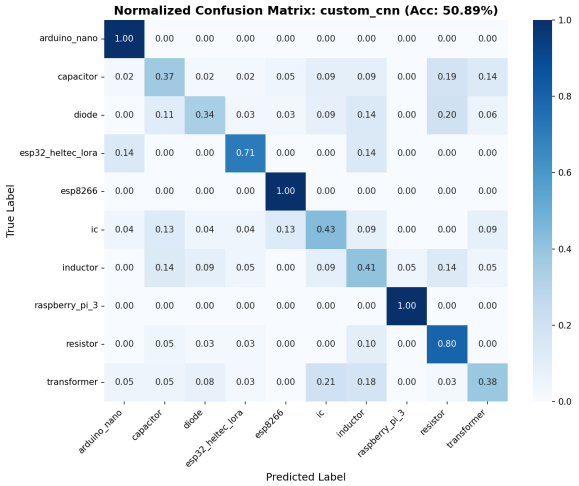


Figure 9: Row-normalised test confusion matrix: custom CNN.

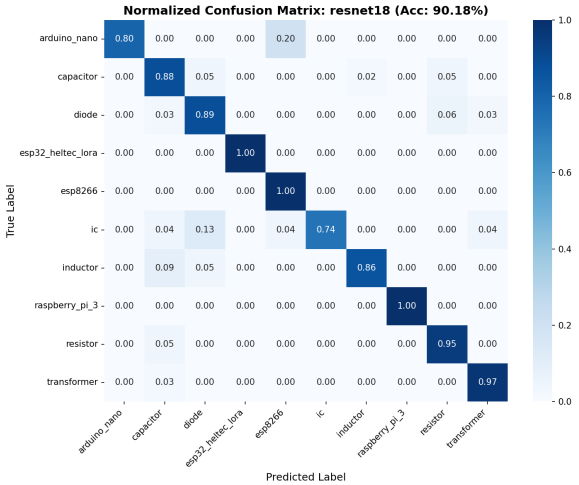


Figure 10: Row-normalised test confusion matrix: ResNet-18.

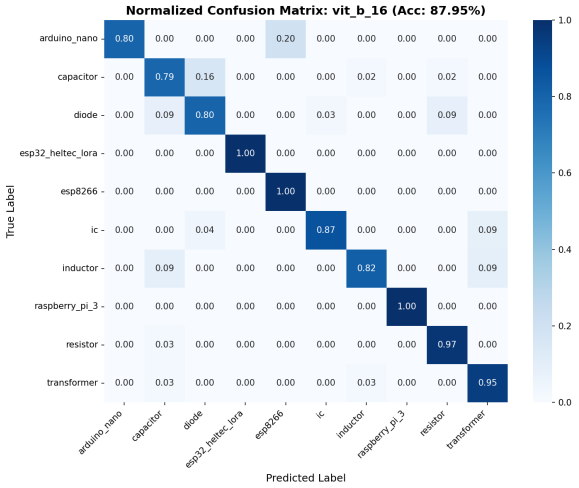


Figure 11: Row-normalised test confusion matrix: ViT-B/16.

Table 6: Most frequent test confusions (true → predicted, count).

Model	Top confusions
Custom CNN (scratch)	capacitor→resistor (8), transformer→ic (8), diode→resistor (7), transformer→inductor (7), capacitor→transformer (6)
ResNet-18 (transfer)	ic→diode (3), capacitor→diode (2), capacitor→resistor (2), diode→resistor (2), inductor→capacitor (2)
ViT-B/16 (transfer)	capacitor→diode (7), diode→capacitor (3), diode→resistor (3), ic→transformer (2), inductor→capacitor (2)

The two pretrained models make few errors (22 of 224 for ResNet-18, 27 for ViT-B/16), and these are concentrated among the small two-lead components. For ViT-B/16 the dominant confusion is `capacitor→diode` (7 images), with `diode→capacitor` and `diode→resistor` (3 each). ResNet-18 spreads its errors more evenly over the same group (`ic→diode` 3, and 2 each for capacitor/diode/resistor/inductor pairs). Of the two hypotheses from Section 1, the board pair Nano vs. ESP8266 is confused only once by each pretrained model, and inductor vs. transformer three times by ViT-B/16 and not at all by ResNet-18, so neither is a major source of error; the real difficulty is the capacitor/diode/resistor group. The custom CNN confuses many more pairs, e.g. `capacitor→resistor` (8), `transformer→ic` (8) and `transformer→inductor` (7). It also predicts a board class for 17 of the 202 component test images, while the pretrained models do so only once in total.

6 Discussion and error analysis

6.1 Correct and incorrect predictions

Figures 12 and 13 show correctly classified test images (one per class) and the misclassifications with the highest confidence for the best model, ResNet-18 (transfer); the same figures for the other models are in `outputs/figures/` and the notebook.

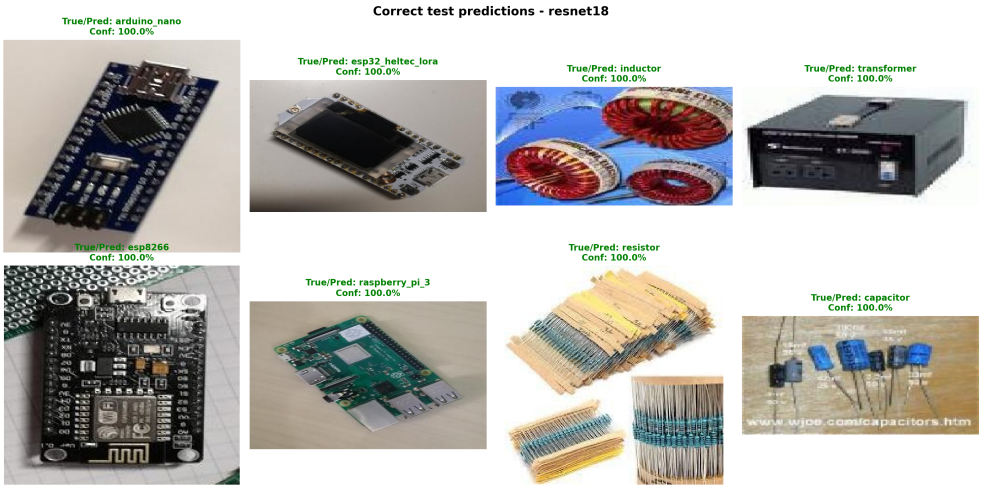


Figure 12: Correct test predictions of ResNet-18 (transfer) with confidence.

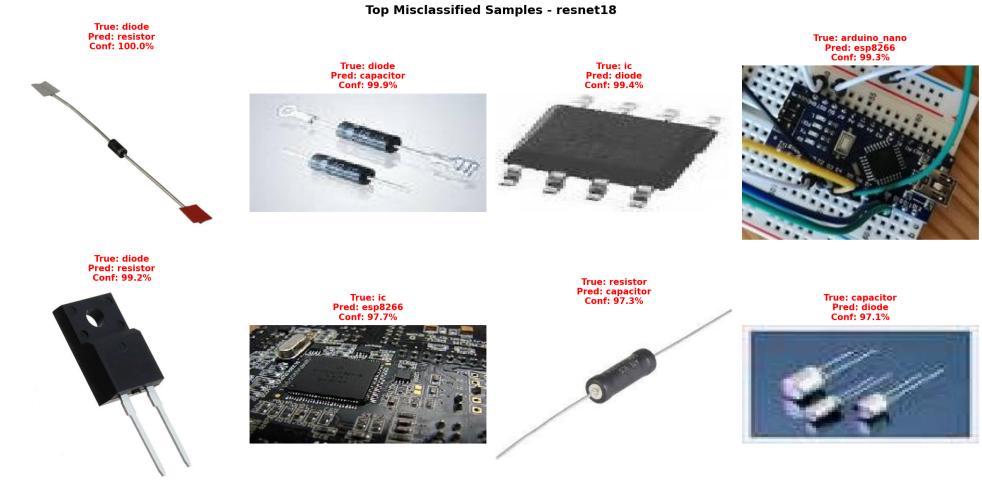


Figure 13: Most confident test errors of ResNet-18 (transfer) (true class, predicted class, confidence).

Inspecting all misclassified test images of the two pretrained models gives four recurring causes. (1) *Look-alike packages.* Many diodes, capacitors and resistors are small dark cylinders with two axial or radial leads; when the colour bands or markings are not visible the shape alone is ambiguous. Examples are an axial power resistor predicted as a capacitor and axial diodes predicted as resistors, both with more than 96% confidence. (2) *Uncommon variants and probable label noise.* Some test images show parts that a human would also hesitate over: a diode in a TO-220 transistor-style package (predicted resistor, 99% confidence), high-voltage diodes with eyelet terminals that look like capacitors, small metal cans with leads labelled `capacitor` that resemble crystals or transistors, and box-shaped film capacitors predicted as diodes. These are labelled correctly according to the catalogue category but are atypical of their class. (3) *Remaining scene images.* One `ic` test image shows a chip on a crowded circuit board, which ResNet-18 calls `esp8266` (98%): the model reacts to the board context rather than to the chip. (4) *Unusual background.* The only board error of ResNet-18 is an Arduino Nano on a breadboard surrounded by wires, predicted as ESP8266 (99%); the Nano training images show the board mostly on plain surfaces. Notably, almost all errors are made with high confidence (>90%), so the softmax confidence of these models would not be a reliable signal for rejecting uncertain predictions without calibration.

6.2 Why the models differ

Transfer learning is the decisive factor. Both pretrained models beat the scratch-trained CNN by more than 35 points of macro-F1, and they do so from the second epoch on. The CNN's training loss stays above 1.0 and its training accuracy at 54%, i.e. it underfits: about 1,400 training images are not enough to learn general visual features (edges, textures, shapes) from scratch, whereas the pretrained backbones bring these features from 1.28 M ImageNet images [21, 9]. The CNN's errors also reveal a shortcut: it predicts board classes for component images with very high confidence (e.g. an inductor as `raspberry_pi_3` at 99%), consistent with relying on photographic style and background, which differ systematically between the board and component sources.

CNN versus transformer. ResNet-18 and ViT-B/16 reach practically the same accuracy (the difference is five test images and reverses on the validation set). ViT-B/16 has 7.7× more parameters and weaker built-in spatial priors [16], which could have hurt on a small dataset; that it still matches ResNet-18 shows that ImageNet pretraining compensates for the missing convolutional inductive bias. Its errors are slightly more concentrated on capacitor/diode, while ResNet-18 is better on those classes and ViT-B/16 better on `ic` (F1 0.91 vs. 0.85), which may reflect the transformer's global view of large multi-pin packages. With a single run per model we cannot claim either architecture is better in general.

Class imbalance and data quality. The class-weighted loss worked as intended: the small board classes are not neglected (they are in fact the easiest). Remaining errors are driven by data quality instead: atypical catalogue images and visually ambiguous packages in the component classes, which our manual review reduced but could not remove entirely.

Cost. ResNet-18 is 7.7× smaller (11.2 M vs. 85.8 M parameters) and about 8× faster at inference (1.4 vs. 12.1 ms per image on a T4 GPU) than ViT-B/16, for equal accuracy.

6.3 Limitations

- **Dataset size and diversity.** With 1902 images the test set is small, so differences of a few images between models are within noise; we report a single seed and did not compute confidence intervals or repeat training with several seeds. **Source confound:** all board images come from one photographer's desk and all component images from web catalogues, so a model could partly separate boards from components by background and photographic style instead of the object. The board classes most likely each show only one or a few physical boards, so test crops can show the same board as training crops, which makes board accuracy optimistic.
- **Group information.** Leakage prevention is only as good as the group ids. Near-duplicate hashing catches resized and recompressed copies, but not different photographs of the same physical board taken from different angles unless the group encodes them.
- **Single-object images.** The system classifies one dominant object per image. Real circuit boards hold many components and would need an object detector upstream.
- **Class definitions.** `ic` and the board classes overlap semantically, because every board carries ICs, and some catalogue `ic` images show whole circuit boards. Only four board types are covered; popular boards such as the Arduino Uno and Raspberry Pi Pico are missing because no openly licensed photographs were found.
- **Additional boards considered but not used.** A collage of 37 further development-board images (including Arduino Uno, Mega and Pro Mini, NodeMCU, ESP32, Raspberry Pi 4 and STM32 Blue Pill) was cut into single images and kept in `data/extra_boards_not_used/`. They were excluded from all experiments: the collage has no stated source or licence, its labels and details are garbled in places so it appears to be synthetic rather than photographed, it gives only one image per board, and only the Nano and NodeMCU ESP8266 match our

classes. Using them would have misrepresented the dataset, so none of the reported results involve them. Real photographs of additional physical boards remain the main next step.

- **Hyperparameters.** Learning rates were set from common practice rather than an extensive search for every model, which could favour one family.

7 Conclusion and recommendation

We built a reproducible pipeline for classifying 10 microcontroller-board and component classes, with explicit cleaning, near-duplicate detection and a group-aware split, and compared a custom CNN, ResNet-18 and ViT-B/16 under one protocol. ResNet-18 (transfer) achieved the highest macro-F1 (0.901) on the held-out test set. We recommend **ResNet-18**: it achieved the best test accuracy (90.2%) and macro-F1 (0.901), it is statistically indistinguishable from ViT-B/16 in accuracy but 7.7× smaller and about 8× faster, and it trained in under five minutes on a free cloud GPU. Transfer learning, not the choice between convolution and attention, made the difference on this small dataset. Concrete next steps are: (1) photograph several different physical boards per class, and add boards such as the Arduino Uno and Raspberry Pi Pico, so that board performance is measured on unseen boards; (2) collect component photographs on a workbench, not only catalogue images, to remove the source confound; (3) repeat training with several seeds and report confidence intervals; and (4) calibrate the confidence scores before using them to reject uncertain predictions.

Group contributions

Carlo Barnardo designed the original project and topic, the Python package structure, the three model wrappers, the first training and evaluation code, and the first report and presentation outline. **Itumeleng Ntela** made the pipeline run end to end, replaced the synthetic placeholder data with the public datasets, carried out the visual review of all images, implemented the duplicate detection and group-aware split, replaced MobileNetV3 by ViT-B/16, trained and evaluated all models on Google Colab, and extended the report, notebook and slides. **Karl Emmanuel Edoun** supplied the collage of additional development-board images, had it cut into individual images, and evaluated it for inclusion; the images were not used for training (see Section 6.3). He also added the helper script `src/mlproject/data/ingest_images.py` for adding new photographs in the required naming format, and completed the group details.

Use of AI tools. An AI assistant (Claude, Anthropic) was used during the project for coding help, the first visual review of the dataset, drafting parts of the report, cutting the extra-board collage into single images, and writing the image-ingestion script. All experiments reported here were run by the group on Google Colab, and the group is responsible for the content.

References

[1] G. Tanner, "Tensorflow-Object-Detection-API-Train-Model: microcontroller detection dataset," GitHub, 2019, MIT licence. [Online]. Available: `https://github.com/TannerGilbert/Tensorflow-Object-Detection-API-Train-Model`

[2] P. Faraji, "Electronic-Components-Classification," GitHub repository. [Online]. Available: `https://github.com/pouria-faraji/Electronic-Components-Classification`

[3] aryaminus, "Electronic components and devices," Kaggle dataset. [Online]. Available: `https://www.kaggle.com/datasets/aryaminus/electronic-components`

[4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE CVPR*, 2009, pp. 248–255.

[5] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[6] P. Goyal *et al.*, "Accurate, large minibatch SGD: Training ImageNet in 1 hour," arXiv:1706.02677, 2017.

[7] K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[8] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, 2015, pp. 448–456.

[9] S. Kornblith, J. Shlens and Q. V. Le, "Do better ImageNet models transfer better?" in *Proc. IEEE CVPR*, 2019, pp. 2661–2671.

[10] N. Krawetz, "Kind of like that," *The Hacker Factor Blog*, 2013. [Online]. Available: `https://www.hackerfactor.com/blog/index.php?/archives/529-Kind-of-Like-That.html`

[11] A. Krizhevsky, I. Sutskever and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in NeurIPS*, 2012, pp. 1097–1105.

[12] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, 2017.

[13] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[14] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in NeurIPS*, 2019, pp. 8024–8035.

[15] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[16] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang and A. Dosovitskiy, "Do vision transformers see like convolutional neural networks?" in *Advances in NeurIPS*, 2021.

[17] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[18] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit and L. Beyer, "How to train your ViT? Data, augmentation, and regularization in vision transformers," *Trans. Mach. Learn. Res.*, 2022.

[19] TorchVision maintainers and contributors, "TorchVision: PyTorch's computer vision library," 2016. [Online]. Available: `https://github.com/pytorch/vision`

[20] A. Vaswani *et al.*, "Attention is all you need," in *Advances in NeurIPS*, 2017, pp. 5998–6008.

[21] J. Yosinski, J. Clune, Y. Bengio and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in NeurIPS*, 2014, pp. 3320–3328.

[22] ITRI626 Practical Project Specification: Deep Learning for Image Classification, North-West University, 2026.